ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA ĐIỆN TỬ VIỄN THÔNG / KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

Môn học: Lập trình nhúng (INT3108)

Hệ thống bãi đỗ xe thông minh sử dụng ESP32-C3 và ESP8266

Giảng viên hướng dẫn: [Họ tên giảng viên]
Lớp môn học: [Mã lớp môn học]

Nhóm sinh viên thực hiện

Họ và tên	MSSV
[Họ và tên thành viên 1]	[MSSV]
[Họ và tên thành viên 2]	[MSSV]
[Họ và tên thành viên 3]	[MSSV]
[Họ và tên thành viên 4]	[MSSV]

Hà Nội, tháng 10 năm 2026

Tóm tắt

Báo cáo trình bày thiết kế và hiện thực một hệ thống bãi đỗ xe thông minh quy mô mô hình gồm năm chỗ đỗ, được xây dựng trên ba vi điều khiển Wi-Fi giá rẻ của Espressif. Một bo **ESP32-C3 SuperMini** đóng vai trò máy chủ trung tâm: phát mạng Wi-Fi riêng (SoftAP), nhận dữ liệu cảm biến qua giao thức UDP, quản lý phiên gửi xe bằng thẻ RFID và tính phí, điều khiển hai thanh chắn (servo SG90), còi báo cháy, hiển thị trạng thái trên màn hình TFT ST7789 240×240 và phục vụ một trang web giám sát (dashboard) cùng API JSON. Hai bo **ESP8266** làm nút cảm biến: một bo đọc 5 cảm biến hồng ngoại (IR) phát hiện xe, cảm biến lửa và cảm biến mực nước; bo còn lại đọc hai đầu đọc RFID MFRC522 (cổng vào và cổng ra) cùng cảm biến nhiệt độ DHT11.

Firmware máy chủ được viết bằng C trên **ESP-IDF 6.1** với hệ điều hành thời gian thực **FreeRTOS**, chia thành các tác vụ (task) độc lập có mức ưu tiên khác nhau; firmware các nút cảm biến viết bằng C++ trên nền Arduino cho ESP8266. Báo cáo đi sâu vào lý do lựa chọn từng linh kiện, từng công nghệ, từng chân GPIO và từng thông số (tần số PWM, độ phân giải, thời gian chống rung, chu kỳ gửi, tốc độ SPI, kích thước bộ đệm, …), kèm theo phân tích bộ nhớ, kết quả kiểm thử và đánh giá hạn chế. Bộ kiểm thử đơn vị chạy trên máy tính cho các module phân tích gói tin và tính phí đạt 26/26 phép kiểm tra.

Từ khóa: ESP32-C3, ESP8266, FreeRTOS, ESP-IDF, UDP, SoftAP, RFID MFRC522, LEDC PWM, ST7789, SPI DMA, NVS, hệ thống nhúng.

Mục lục

Tóm tắt **i**

Danh mục từ viết tắt **viii**

1 Giới thiệu **1**
1.1 Bối cảnh và động lực 1
1.2 Mục tiêu đề tài 1
1.3 Phạm vi 1
1.4 Cấu trúc báo cáo 2

2 Phân tích yêu cầu **3**
2.1 Yêu cầu chức năng 3
2.2 Yêu cầu phi chức năng 3

3 Kiến trúc tổng thể **5**
3.1 Sơ đồ khối hệ thống 5
3.2 Phân chia vai trò 6
3.3 Luồng hoạt động chính 6

4 Lựa chọn phần cứng **8**
4.1 Vi điều khiển máy chủ: ESP32-C3 SuperMini 8
4.2 Vi điều khiển nút cảm biến: ESP8266 8
4.3 Màn hình TFT ST7789 1.54 inch 240×240 9
4.4 Servo SG90 cho thanh chắn 9
4.5 Cảm biến 9
4.6 Bảng phân chân và lý do chọn chân 10
4.6.1 Máy chủ ESP32-C3 SuperMini 10
4.6.2 Nút cảm biến ESP8266 (Sensor Client) 10
4.6.3 Nút RFID ESP8266 (RFID Client) 11
4.7 Nguồn điện 11

5 Lựa chọn công nghệ phần mềm **12**
5.1 ESP-IDF + FreeRTOS cho máy chủ, Arduino cho nút cảm biến 12
5.2 Mạng: Wi-Fi SoftAP do máy chủ phát 12
5.3 Giao thức truyền: UDP broadcast 13
5.4 Định dạng bản tin: JSON 13
5.5 Giao diện web nhúng trong firmware 14

5.6 Lưu trữ: NVS 14

6 Thiết kế firmware máy chủ (ESP32-C3) 15
6.1 Cấu trúc mã nguồn 15
6.2 Trình tự khởi động 15
6.3 Các tác vụ FreeRTOS và đồng bộ 16
6.4 Module trạng thái bãi xe 17
6.5 Bộ phân tích JSON tự viết 17
6.6 Phiên gửi xe RFID và tính phí 18
6.7 Điều khiển thanh chắn bằng LEDC PWM 19
6.8 Còi báo cháy 21
6.9 Màn hình ST7789 21
6.9.1 Cấu hình SPI 21
6.9.2 Framebuffer và định dạng màu 21
6.9.3 Chỉ vẽ lại khi cần 22
6.9.4 Hoạt hình GIF nén LZSS 22
6.10 UDP server 23
6.11 HTTP server và API 24

7 Thiết kế firmware nút cảm biến (ESP8266) 25
7.1 Sensor Client: cảm biến chỗ đỗ, lửa và nước 25
7.1.1 Vòng lặp không chặn 25
7.1.2 Chống rung (debounce) 250 ms 25
7.1.3 Logic tích cực mức thấp và cảm biến lửa 26
7.1.4 Quy đổi mực nước 26
7.1.5 Đóng gói và gửi 26
7.2 RFID Client: hai đầu đọc RFID và DHT11 27
7.2.1 Ghép hai MFRC522 trên một bus SPI 27
7.2.2 Cảm biến nhiệt độ DHT11 28
7.2.3 Nạp firmware qua mạng (OTA) 28

8 Giao thức truyền thông và giao diện web 29
8.1 Thông số mạng 29
8.2 Bản tin UDP 29
8.3 HTTP API 30
8.4 Dashboard 30

9 Phân tích tài nguyên 31
9.1 Bộ nhớ RAM máy chủ 31
9.2 Bộ nhớ flash máy chủ 31

9.3 Thời gian và băng thông 31

10 Kiểm thử **33**

10.1 Kiểm thử đơn vị trên máy tính 33

10.2 Kiểm thử tích hợp bằng công cụ giả lập 34

10.3 Kịch bản kiểm thử hệ thống 34

11 Đánh giá, hạn chế và hướng phát triển **36**

11.1 Kết quả đạt được 36

11.2 Hạn chế 36

11.3 Hướng phát triển 36

Kết luận **38**

Tài liệu tham khảo **39**

A Hướng dẫn biên dịch và nạp chương trình **40**

B Phân công công việc **41**

Danh sách hình vẽ

3.1 Sơ đồ khối tổng thể của hệ thống 5
3.2 Biểu đồ tuần tự xe vào và ra bằng thẻ RFID 7

6.1 Máy trạng thái của một thẻ RFID 18
6.2 Dạng xung điều khiển servo SG90 19
6.3 Bố cục giao diện màn hình TFT (mô phỏng) 22

7.1 Nguyên lý chống rung: chỉ chấp nhận mức mới khi giữ ổn định đủ 250 ms . . . 25

Danh sách bảng

2.1 Yêu cầu chức năng 3
2.2 Yêu cầu phi chức năng 3

3.1 Vai trò của từng nút 6

4.1 So sánh các phương án vi điều khiển cho máy chủ 8
4.2 Các cảm biến và lý do lựa chọn 9
4.3 Phân chân ESP32-C3 10
4.4 Phân chân Sensor Client 10
4.5 Phân chân RFID Client 11

5.1 So sánh giao thức truyền dữ liệu cảm biến 13

6.1 Các tác vụ FreeRTOS trên máy chủ 16
6.2 Hiệu quả nén khung hình 22

8.1 Thông số mạng 29
8.2 Đặc tả trường dữ liệu 29
8.3 Các endpoint HTTP 30

9.1 Các vùng nhớ lớn do ứng dụng cấp phát (ước tính từ mã nguồn) 31

10.1 Các trường hợp kiểm thử đơn vị 33
10.2 Kịch bản kiểm thử trên phần cứng và hành vi mong đợi 34

B.1 Phân công công việc (nhóm điền) 41

Danh mục mã nguồn

6.1 Cây thư mục firmware máy chủ 15
6.2 Hàm `app_main` (`main.c`) 15
6.3 Cấu trúc trạng thái dùng chung (`parking_state.h`) 17
6.4 Đặt góc servo và ánh xạ trạng thái (`servo.c`) 20
6.5 Vòng lặp chính của UDP server (rút gọn) 23
7.1 Thuật toán chống rung cho cảm biến IR (`SensorClient.ino`) 25
7.2 Tạo bản tin `parking_update` 26
7.3 Quét luân phiên hai đầu đọc (`RFIDClient.ino`) 27
8.1 Bản tin `parking_update` (Sensor Client → Server, mỗi 1 s) 29
8.2 Bản tin `rfid_event` (RFID Client → Server, mỗi lần quẹt thẻ) 29

Danh mục từ viết tắt

Viết tắt	Ý nghĩa
ADC	Analog-to-Digital Converter — bộ chuyển đổi tương tự–số
AP / SoftAP	Access Point — điểm truy cập Wi-Fi do chính vi điều khiển phát ra
API	Application Programming Interface
DMA	Direct Memory Access — truy cập bộ nhớ trực tiếp, không qua CPU
GPIO	General Purpose Input/Output
HTTP	HyperText Transfer Protocol
IR	Infrared — hồng ngoại
ISR	Interrupt Service Routine
JSON	JavaScript Object Notation
LEDC	LED Control — khối phát PWM phần cứng của ESP32
LZSS	Lempel–Ziv–Storer–Szymanski — thuật toán nén từ điển
NVS	Non-Volatile Storage — vùng lưu trữ khóa–giá trị trên flash của ESP-IDF
OTA	Over-The-Air — nạp firmware qua mạng
PWM	Pulse Width Modulation — điều chế độ rộng xung
RFID	Radio Frequency Identification
RTOS	Real-Time Operating System
SPI	Serial Peripheral Interface
SSID	Service Set Identifier — tên mạng Wi-Fi
TFT	Thin-Film Transistor (màn hình)
UDP	User Datagram Protocol
UID	Unique Identifier — mã định danh của thẻ RFID

Chương 1. Giới thiệu

1.1 Bối cảnh và động lực

Tại các đô thị lớn như Hà Nội, việc tìm chỗ đỗ xe tiêu tốn nhiều thời gian và gây ùn tắc cục bộ. Các bãi xe truyền thống quản lý bằng vé giấy, ghi chép thủ công, không cho biết trước còn bao nhiêu chỗ trống và thiếu cơ chế cảnh báo an toàn (cháy nổ, ngập nước tầng hầm). Một hệ thống bãi đỗ xe thông minh giải quyết các vấn đề này bằng cách tự động phát hiện chỗ trống, kiểm soát ra vào bằng thẻ, tính phí tự động và giám sát môi trường theo thời gian thực.

Đây là một bài toán điển hình của *lập trình nhúng*: nhiều cảm biến và cơ cấu chấp hành, tài nguyên phần cứng hạn chế (vài trăm KB RAM), yêu cầu phản hồi nhanh, chạy liên tục không có người giám sát và giao tiếp không dây giữa nhiều nút.

1.2 Mục tiêu đề tài

1. Phát hiện trạng thái trống/có xe của 5 chỗ đỗ bằng cảm biến hồng ngoại.
2. Kiểm soát xe ra vào bằng thẻ RFID: quẹt thẻ lần đầu mở thanh chắn cổng vào, quẹt lại cùng thẻ mở thanh chắn cổng ra; thanh chắn tự đóng sau 3 giây.
3. Tính phí gửi xe tự động theo thời gian (5.000 VNĐ cho mỗi giờ bắt đầu, tối thiểu 1 giờ) và hiển thị hóa đơn trên màn hình.
4. Giám sát môi trường: phát hiện lửa (kèm còi và màn hình cảnh báo), đo mực nước, đo nhiệt độ.
5. Cung cấp giao diện web giám sát và điều khiển thanh chắn từ điện thoại/máy tính, không cần Internet hay ứng dụng cài đặt.
6. Toàn bộ hệ thống hoạt động độc lập, tự tạo mạng riêng, khởi động lại vẫn giữ trạng thái thanh chắn.

1.3 Phạm vi

Hệ thống là **mô hình** quy mô phòng thí nghiệm (5 chỗ đỗ, 2 cổng). Các vấn đề như thanh toán điện tử, cơ sở dữ liệu đám mây, nhận dạng biển số không nằm trong phạm vi; chúng được đề cập ở phần hướng phát triển (Chương 11).

1.4 Cấu trúc báo cáo

Chương 2 phân tích yêu cầu. Chương 3 trình bày kiến trúc tổng thể. Chương 4 và 5 giải thích lựa chọn phần cứng và công nghệ phần mềm. Chương 6 mô tả chi tiết firmware máy chủ; Chương 7 mô tả firmware hai nút cảm biến. Chương 8 đặc tả giao thức truyền thông và giao diện web. Chương 9 phân tích tài nguyên. Chương 10 trình bày kiểm thử, Chương 11 đánh giá và hướng phát triển.

Chương 2. Phân tích yêu cầu

2.1 Yêu cầu chức năng

Bảng 2.1: Yêu cầu chức năng

Mã	Mô tả
FR1	Đọc trạng thái 5 chỗ đỗ, loại bỏ nhiễu/rung tín hiệu, cập nhật về trung tâm định kỳ.
FR2	Hiển thị số chỗ trống và trạng thái từng chỗ trên màn hình tại bãi và trên web.
FR3	Nhận mã UID thẻ RFID tại cổng; lần quẹt đầu là *vào*, lần quẹt kế tiếp của cùng thẻ là *ra*.
FR4	Mở thanh chắn tương ứng khi có sự kiện vào/ra; tự đóng sau 3 giây.
FR5	Tính phí khi xe ra: làm tròn lên theo giờ, tối thiểu 1 giờ, 5.000 VNĐ/giờ; hiển thị UID, thời gian gửi, số tiền trong 60 giây.
FR6	Khi cảm biến lửa kích hoạt: hiển thị màn hình cảnh báo động, bật còi, hiện băng rôn cảnh báo trên web.
FR7	Đo và hiển thị mực nước (mm) và nhiệt độ (°C).
FR8	Cho phép mở/đóng từng thanh chắn và mở/đóng khẩn cấp đồng thời cả hai từ web.
FR9	Lưu trạng thái thanh chắn vào bộ nhớ không mất dữ liệu và khôi phục khi khởi động lại.
FR10	Từ chối từ xe mới khi đã có 5 phiên gửi xe đang hoạt động (bãi đầy).

2.2 Yêu cầu phi chức năng

Bảng 2.2: Yêu cầu phi chức năng

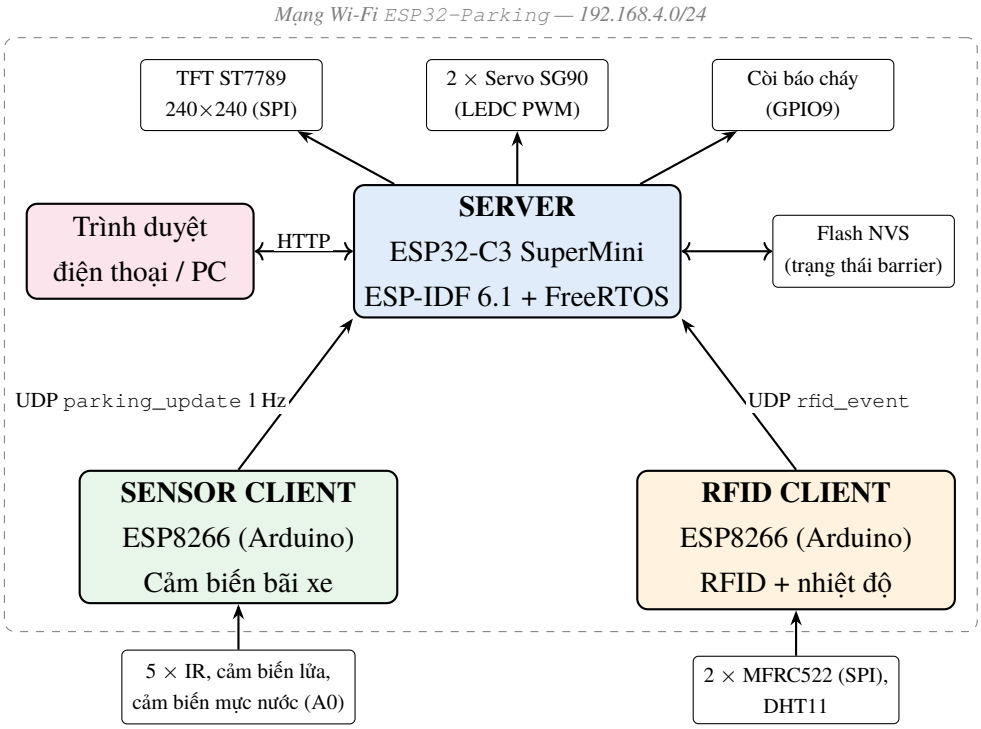
Mã	Mô tả
NFR1	**Độ trễ**: thay đổi chỗ đỗ hiển thị trong khoảng ≤ 2 s; thanh chắn phản hồi thẻ RFID dưới 1 s.
NFR2	**Độc lập hạ tầng**: không phụ thuộc router hay Internet.
NFR3	**Bền vững**: firmware chạy liên tục, tự phục hồi khi socket lỗi, mất Wi-Fi.
NFR4	**An toàn đầu vào**: mọi gói tin từ mạng phải được kiểm tra chặt chẽ, không tràn bộ đệm, không cấp phát động khi phân tích.

NFR5	**Tài nguyên**: vừa trong 400 KB SRAM và 2 MB flash của ESP32-C3 SuperMini.
NFR6	**Chi phí thấp**: linh kiện phổ thông, tổng chi phí dưới vài trăm nghìn đồng.
NFR7	**Dễ bảo trì**: mã nguồn chia module, có công cụ giả lập để kiểm thử không cần phần cứng.

Chương 3. Kiến trúc tổng thể

3.1 Sơ đồ khối hệ thống

Hệ thống gồm ba nút phần cứng và các thiết bị người dùng (điện thoại, máy tính) cùng kết nối vào một mạng Wi-Fi cục bộ do máy chủ phát (Hình 3.1).



Hình 3.1: Sơ đồ khối tổng thể của hệ thống

3.2 Phân chia vai trò

Bảng 3.1: Vai trò của từng nút

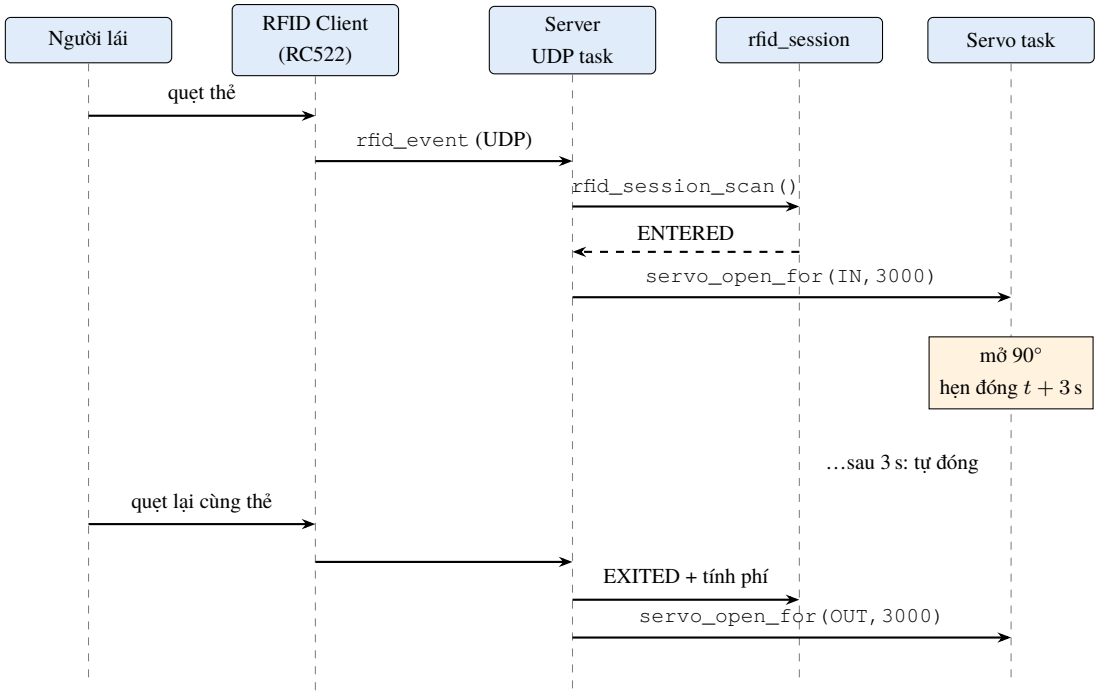
Nút	Nhiệm vụ	Mã nguồn
Server	Phát Wi-Fi, nhận UDP, quản lý phiên RFID + tính phí, điều khiển barrier, còi, màn hình, web server	`Server/main/`
Sensor Client	Đọc 5 IR + lửa + nước, chống rung, gửi trạng thái đầy đủ mỗi 1 s	`SensorClient/SensorClient.ino`
RFID Client	Quét luân phiên 2 đầu đọc RFID, đọc DHT11, gửi sự kiện mỗi lần quẹt thẻ	`RFIDClient/RFIDClient.ino`
Trình duyệt	Hiển thị dashboard, gọi API điều khiển barrier	`Server/main/network/http/`

Tại sao tách thành 3 bo mạch thay vì 1. (1) *Giới hạn chân GPIO*: ESP32-C3 SuperMini chỉ có khoảng 13 chân GPIO dùng được; màn hình đã chiếm 6 chân, 2 servo và còi chiếm 3 chân. Năm cảm biến IR, cảm biến lửa, cảm biến nước, hai đầu đọc RFID (mỗi đầu cần chân SS riêng + bus SPI) và DHT11 cần thêm hơn 12 chân — không thể đặt chung. (2) *Phân bố vật lý*: trong thực tế cảm biến chỗ đỗ nằm trong bãi, đầu đọc RFID nằm ở cổng, bộ điều khiển trung tâm đặt ở phòng bảo vệ; kết nối không dây giúp không phải kéo dây dài. (3) *Cô lập lỗi*: một nút cảm biến bị treo không làm dừng màn hình hay thanh chắn. (4) *Phân công nhóm*: mỗi thành viên phát triển, nạp và kiểm thử một bo độc lập.

3.3 Luồng hoạt động chính

Luồng cập nhật chỗ đỗ. Sensor Client đọc cảm biến liên tục trong `loop()`, lọc nhiễu 250 ms, cứ mỗi 1 s đóng gói *toàn bộ trạng thái* thành JSON và gửi broadcast UDP. Server nhận, kiểm tra cú pháp, ghi vào biến trạng thái dùng chung (bảo vệ bởi critical section), bật/tắt còi. Tác vụ màn hình và trang web đọc lại trạng thái này để hiển thị.

Luồng xe vào/ra bằng RFID. Hình 3.2 mô tả trình tự.



Hình 3.2: Biểu đồ tuần tự xe vào và ra bằng thẻ RFID

Chương 4. Lựa chọn phần cứng

4.1 Vi điều khiển máy chủ: ESP32-C3 SuperMini

Bảng 4.1: So sánh các phương án vi điều khiển cho máy chủ

Tiêu chí	ESP32-C3	ESP32 (classic)	ESP8266	Arduino Uno + module Wi-Fi
CPU	RISC-V 32-bit, 160 MHz, 1 nhân	Xtensa LX6, 240 MHz, 2 nhân	Xtensa L106, 80/160 MHz	AVR 8-bit, 16 MHz
SRAM	400 KB	520 KB	∼80 KB khả dụng	2 KB
Wi-Fi/BLE	802.11 b/g/n + BLE 5	b/g/n + BT/BLE 4.2	b/g/n	cần module ngoài
PWM phần cứng	LEDC 6 kênh, 14 bit	LEDC 16 kênh	PWM mềm	6 kênh 8 bit
SPI DMA	Có (GDMA)	Có	Hạn chế	Không
RTOS chính thức	FreeRTOS (ESP-IDF)	FreeRTOS	Non-OS / RTOS SDK	Không
Kích thước bo	22×18 mm	$\sim 50 \times 28$ mm	$\sim 48 \times 26$ mm	lớn

Lý do lựa chọn. Máy chủ phải *đồng thời* chạy Wi-Fi AP, web server, UDP server, vẽ màn hình 240×240 màu 16 bit và tạo PWM cho servo. Màn hình cần một framebuffer $240 \times 240 \times 2 = 115\,200$ byte — vượt xa toàn bộ RAM của ESP8266 hay Arduino. ESP32-C3 có 400 KB SRAM, có DMA cho SPI để đẩy dữ liệu màn hình mà CPU không phải chờ, có khối LEDC tạo PWM phần cứng chính xác (servo không bị giật khi CPU bận xử lý mạng), và được hỗ trợ chính thức bởi ESP-IDF/FreeRTOS. So với ESP32 classic, C3 rẻ hơn, nhỏ gọn hơn (bản SuperMini chỉ 22×18 mm) và một nhân 160 MHz đã thừa năng lực cho bài toán này. Kiến trúc RISC-V cũng là một điểm học thuật thú vị.

4.2 Vi điều khiển nút cảm biến: ESP8266

Lý do lựa chọn. Nút cảm biến chỉ cần đọc vài chân số, một chân ADC, giao tiếp SPI với RC522 và gửi một gói UDP nhỏ — nhu cầu tính toán rất thấp. ESP8266 (NodeMCU/Wemos D1) là bo Wi-Fi rẻ nhất, phổ biến nhất trên thị trường Việt Nam, nhóm đã có sẵn, có chân ADC A0 (ESP32-C3 cũng có ADC nhưng chân đã dùng hết), và có hệ sinh thái thư viện Arduino rất lớn (`MFRC522`, `DHT`, `ArduinoOTA`) giúp rút ngắn thời gian phát triển.

4.3 Màn hình TFT ST7789 1.54 inch 240×240

Lý do lựa chọn. Màn hình màu cho phép thể hiện trực quan (xanh = trống, đỏ = có xe, màn hình đỏ nhấp nháy khi cháy) — điều mà LCD1602 ký tự không làm được. ST7789 dùng giao tiếp SPI chỉ ghi (không cần MISO) nên chỉ tốn 6 chân; độ phân giải 240×240 vuông vừa đủ chi tiết nhưng framebuffer 115 KB vẫn vừa RAM của C3. ESP-IDF có sẵn driver `esp_lcd_panel_st7789`, không cần thư viện ngoài. So với OLED SSD1306 (128×64, đơn sắc), ST7789 hiển thị được nhiều thông tin hơn và có màu.

4.4 Servo SG90 cho thanh chắn

Lý do lựa chọn. SG90 rẻ, nhẹ (9 g), mô-men khoảng 1,8 kg·cm — dư sức nâng một thanh chắn bằng nhựa/que nhẹ của mô hình. Servo có mạch phản hồi vị trí bên trong nên chỉ cần cấp xung PWM 50 Hz là giữ được góc, không cần cảm biến vị trí hay mạch cầu H như động cơ DC. Hai servo được dùng riêng cho cổng vào và cổng ra để xe vào và ra không phải dùng chung một làn.

4.5 Cảm biến

Bảng 4.2: Các cảm biến và lý do lựa chọn

Cảm biến	Ngõ ra	Lý do lựa chọn
IR phản xạ (TCRT5000/FC-51) $\times 5$	Số, tích cực mức thấp	Rẻ, phát hiện vật cản ở cự ly vài cm phù hợp mô hình xe; có so sánh LM393 và biến trở chỉnh ngưỡng trên module nên vi điều khiển chỉ cần đọc mức logic. Siêu âm HC-SR04 chính xác hơn nhưng cần 2 chân/cảm biến và đo thời gian, tốn tài nguyên cho 5 chỗ.
Cảm biến lửa (IR 760–1100 nm)	Số (DO)	Nhạy với bức xạ hồng ngoại của ngọn lửa, phản ứng nhanh hơn cảm biến khói/nhiệt; module có ngõ ra số sẵn ngưỡng.
Cảm biến mực nước điện trở	Tương tự	Đơn giản, đo được mức ngập 0–40 mm của mô hình tầng hầm, chỉ cần 1 chân ADC.
RFID MFRC522 13,56 MHz $\times 2$	SPI	Đọc thẻ MIFARE phổ biến (thẻ sinh viên, móc khóa), tầm đọc vài cm phù hợp quẹt thẻ tại cổng; thư viện Arduino ổn định; giá rẻ.
DHT11	1-wire riêng	Đo nhiệt độ 0–50 °C, sai số ± 2 °C — đủ cho giám sát môi trường; chỉ cần 1 chân.
Còi (buzzer module)	—	Báo động âm thanh tại chỗ khi có cháy, điều khiển bằng 1 chân số.

4.6 Bảng phân chân và lý do chọn chân

4.6.1 Máy chủ ESP32-C3 SuperMini

Bảng 4.3: Phân chân ESP32-C3

Tín hiệu	GPIO	Ghi chú / lý do
TFT SCK	4	Chân thường, ghép vào SPI2 qua GPIO matrix
TFT MOSI (SDA)	6	Chân thường; màn hình không cần MISO
TFT DC	7	Chọn lệnh/dữ liệu
TFT RST	3	Reset phần cứng màn hình khi khởi động
TFT CS	10	Chip select
TFT BL	5	Đèn nền: tắt trong lúc khởi tạo để không thấy rác, bật sau khi vẽ xong
Servo cổng vào	0	Chân thường (chung với XTAL 32 kHz, bo SuperMini không dùng thạch anh này)
Servo cổng ra	1	Như trên
Còi	9	Tích cực mức thấp (xem lưu ý bên dưới)

Khi chọn chân, nhóm **tránh** các chân có chức năng đặc biệt trên ESP32-C3: GPIO18/19 (USB-Serial/JTAG dùng để nạp và xem log), GPIO20/21 (UART0), GPIO12–17 (nối flash SPI nội), GPIO8 (chân strapping, nối LED trên bo). Toàn bộ chân màn hình được khai báo trong `Kconfig.projbuild` nên có thể đổi qua `idf.py menuconfig` mà không sửa mã nguồn.

Lưu ý về GPIO9. GPIO9 là chân *strapping* quyết định chế độ khởi động (mức thấp lúc reset = vào chế độ nạp firmware). Chân này có điện trở kéo lên mặc định nên ở trạng thái nghỉ là mức cao. Nhóm dùng còi loại *tích cực mức thấp*: mức cao = im lặng, nên trạng thái mặc định của chân trùng với trạng thái "còi tắt" — đây là lý do còi được gắn vào GPIO9 và firmware đặt chân lên mức cao ngay khi khởi tạo. Cần đảm bảo module còi không kéo chân xuống thấp lúc cấp nguồn, nếu không bo sẽ vào chế độ nạp.

4.6.2 Nút cảm biến ESP8266 (Sensor Client)

Bảng 4.4: Phân chân Sensor Client

Cảm biến	GPIO	Ghi chú
Chỗ S1–S5	16, 4, 14, 13, 12	Tránh GPIO0, 2, 15 (strapping) và GPIO1, 3 (UART log); GPIO16 không có ngắt nhưng firmware dùng polling nên không ảnh hưởng
Lửa (DO)	5	Chân thường
Nước	A0	Chân ADC duy nhất của ESP8266 (10 bit)

4.6.3 Nút RFID ESP8266 (RFID Client)

Bảng 4.5: Phân chân RFID Client

Tín hiệu	GPIO	Ghi chú
SCK / MOSI / MISO	14 / 13 / 12	SPI phần cứng (HSPI) của ESP8266, dùng chung cho 2 đầu đọc
SS đầu đọc 1 (cổng vào)	4 (D2)	Mỗi thiết bị SPI cần một chân chọn chip riêng
SS đầu đọc 2 (cổng ra)	5 (D1)	
RST (chung)	16 (D0)	Dùng chung reset để tiết kiệm chân
DHT11	2 (D4)	GPIO2 phải ở mức cao khi khởi động; DHT11 có điện trở kéo lên nên phù hợp

4.7 Nguồn điện

Servo SG90 có dòng khởi động/kẹt tới vài trăm mA mỗi con. Nếu cấp từ chân 3V3 của bo, sụt áp sẽ làm ESP32-C3 reset (brown-out) mỗi lần thanh chắn chuyển động. Vì vậy servo được cấp từ nguồn 5 V riêng, **nối chung GND** với ESP32-C3 để tín hiệu PWM có cùng mức tham chiếu. Tín hiệu PWM 3,3 V của ESP32-C3 vẫn được SG90 nhận đúng mức cao.

Chương 5. Lựa chọn công nghệ phần mềm

5.1 ESP-IDF + FreeRTOS cho máy chủ, Arduino cho nút cảm biến

Tại sao máy chủ dùng ESP-IDF thay vì Arduino. Máy chủ có nhiều hoạt động diễn ra song song với chu kỳ khác nhau: nhận UDP (sự kiện bất kỳ lúc nào), vẽ màn hình (500 ms), hoạt hình GIF (100 ms/khung), kiểm tra hẹn giờ đóng thanh chắn (50 ms), phục vụ HTTP. Viết tất cả trong một `loop()` của Arduino buộc phải dùng máy trạng thái thủ công và một thao tác chậm (ví dụ vẽ màn hình) sẽ làm chậm toàn bộ. ESP-IDF cung cấp FreeRTOS với *task* có ưu tiên, *mutex*, *critical section*; cung cấp driver chính thức cho `esp_lcd` (SPI DMA), LEDC, NVS, `esp_http_server`, lwIP socket. Đây cũng là framework được Espressif hỗ trợ dài hạn và là trọng tâm kiến thức của môn lập trình nhúng (RTOS, driver, quản lý bộ nhớ).

Tại sao nút cảm biến dùng Arduino. Logic của nút cảm biến tuần tự và đơn giản; ưu tiên là phát triển nhanh và dùng được thư viện cộng đồng đã kiểm chứng (MFRC522, DHT). Arduino core cho ESP8266 vẫn chạy trên SDK của Espressif nên đủ ổn định, đồng thời có `ArduinoOTA` để nạp lại firmware qua Wi-Fi khi các bo đã gắn cố định vào mô hình.

5.2 Mạng: Wi-Fi SoftAP do máy chủ phát

Lý do lựa chọn. Nếu dùng router có sẵn, hệ thống phụ thuộc vào hạ tầng bên ngoài, địa chỉ IP máy chủ thay đổi theo DHCP và mạng trường thường chặn thiết bị lạ. Khi ESP32-C3 tự phát AP, địa chỉ máy chủ luôn cố định là `192.168.4.1`, các client luôn biết đích đến, hệ thống mang đi demo ở bất kỳ đâu và người dùng chỉ cần kết nối Wi-Fi rồi mở trình duyệt. Các thông số: kênh 1 (một trong ba kênh không chồng lấn 1/6/11 của băng 2,4 GHz); tối đa 4 trạm (2 client + 2 thiết bị xem dashboard; mỗi trạm tốn RAM trong AP nên không đặt lớn hơn); WPA2-PSK với mật khẩu 8 ký tự (độ dài tối thiểu của WPA2); PMF không bắt buộc để tương thích với ESP8266.

5.3 Giao thức truyền: UDP broadcast

Bảng 5.1: So sánh giao thức truyền dữ liệu cảm biến

	UDP	**HTTP/TCP**	**MQTT**
Thiết lập kết nối	Không	Bắt tay 3 bước mỗi request (hoặc giữ kết nối)	Cần broker
Overhead mỗi bản tin	8 byte header	Vài trăm byte header HTTP	2+ byte, nhưng trên TCP
Bộ nhớ phía server	1 socket, 1 bộ đệm 256 B	Mỗi kết nối 1 socket + bộ đệm	Cần chạy broker trên ESP
Khi mất gói	Mất — gói kế tiếp bù lại	Tự truyền lại	Có QoS
Hạ tầng thêm	Không	Không	Broker (Mosquitto…)

Lý do lựa chọn. Dữ liệu chỗ đỗ được gửi dưới dạng **ảnh chụp trạng thái đầy đủ** (full snapshot) mỗi giây, không phải dạng "sự kiện thay đổi". Do đó mất một gói không làm sai trạng thái lâu dài: gói sau 1 s sẽ sửa lại. Với đặc tính đó, độ tin cậy của TCP là không cần thiết, trong khi chi phí của nó (bắt tay, socket, bộ đệm, tác vụ) là đáng kể trên vi điều khiển. Lịch sử phát triển của nhóm cũng phản ánh điều này: nhóm đã thử định dạng request khác rồi quay lại UDP (commit `revert to udp`). Gửi tới địa chỉ *broadcast* `192.168.4.255` giúp client không cần biết IP máy chủ và cho phép một máy tính trong mạng "nghe lén" để gỡ lỗi. Cổng 3333 là cổng không đặc quyền, không trùng dịch vụ phổ biến và là cổng mẫu trong ví dụ UDP của ESP-IDF.

Hạn chế của lựa chọn này (sự kiện RFID không có xác nhận, gói broadcast trên Wi-Fi không được MAC-layer ACK) được phân tích ở Chương 11.

5.4 Định dạng bản tin: JSON

Lý do lựa chọn. JSON đọc được bằng mắt trong log, dễ sinh từ `snprintf` trên ESP8266 và từ Python khi giả lập, và có trường `type` để phân biệt nhiều loại bản tin trên cùng một cổng. Bản tin dài khoảng 60–80 byte, vừa một gói UDP nhỏ (không phân mảnh IP). Định dạng nhị phân sẽ ngắn hơn nhưng khó gỡ lỗi; với tần suất 1 gói/giây, lợi ích băng thông không đáng kể.

5.5 Giao diện web nhúng trong firmware

Lý do lựa chọn. Trang `index.html` và `style.css` được nhúng thẳng vào firmware bằng `EMBED_TXTFILES` của CMake/ESP-IDF; trình liên kết tạo các ký hiệu `_binary_index_html_start/_end`. Cách này không cần phân vùng file (SPIFFS/LittleFS), chỉ cần nạp *một* file `.bin`, và tệp HTML luôn đồng bộ phiên bản với firmware. Trình duyệt có sẵn trên mọi điện thoại nên không cần viết ứng dụng di động.

5.6 Lưu trữ: NVS

Lý do lựa chọn. Chỉ cần lưu hai giá trị boolean (barrier vào/ra đang mở hay đóng). NVS của ESP-IDF là kho khóa–giá trị trên flash có cân bằng hao mòn (wear levelling) và an toàn khi mất điện giữa chừng, phù hợp hơn tự ghi flash thô. NVS cũng đã phải khởi tạo cho Wi-Fi (lưu dữ liệu hiệu chuẩn PHY) nên không tốn thêm phân vùng.

Chương 6. Thiết kế firmware máy chủ (ESP32-C3)

6.1 Cấu trúc mã nguồn

```
Server/main/
|-- main.c                  app_main(): thu tu khoi dong
|-- Kconfig.projbuild       Cau hinh chan TFT (menuconfig)
|-- barrier/servo.c         LEDC PWM, NVS, task tu dong dong
|-- buzzer/buzzer.c         Coi active-low GPIO9
|-- display/lcd_display.c   Giao dien ST7789, man hinh bao chay
|-- display/*_gif.c         Khung hinh GIF nen LZSS + bo giai nen
|-- network/wifi_ap.c       SoftAP
|-- network/http/           Web server + dashboard nhung
|-- network/udp/            UDP server (cong 3333)
`-- parking/
    |-- parking_state.c     Trang thai dung chung (critical section)
    |-- parking_update.c    Bo phan tich JSON parking_update
    `-- rfid_session.c      Phien gui xe RFID + tinh phi
```

Mã nguồn 6.1: Cây thư mục firmware máy chủ

Mỗi thư mục là một module có file `.h` công khai tối thiểu (ví dụ `servo.h` chỉ có 7 hàm). Module mạng không biết chi tiết màn hình, màn hình không biết chi tiết UDP; chúng chỉ giao tiếp qua `parking_state` và `rfid_session`. Thiết kế này giúp kiểm thử độc lập các module `parking/` trên máy tính (Chương 10).

6.2 Trình tự khởi động

```
1  void app_main(void)
2  {
3      esp_err_t ret = nvs_flash_init();
4      if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
5          ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
6          ESP_ERROR_CHECK(nvs_flash_erase());
7          ret = nvs_flash_init();
8      }
9      ESP_ERROR_CHECK(ret);
10     // Start the local screen before Wi-Fi consumes system memory.
11     ESP_ERROR_CHECK(lcd_display_start());
12     ESP_ERROR_CHECK(buzzer_start());
13     ESP_ERROR_CHECK(servo_start());
14     ESP_ERROR_CHECK(network_service_start());
15 }
```

Mã nguồn 6.2: Hàm `app_main` (`main.c`)

Tại sao thứ tự khởi động như vậy.

1. **NVS trước tiên** vì cả Wi-Fi và servo đều đọc NVS. Nếu phân vùng NVS đầy hoặc khác phiên bản (sau khi đổi ESP-IDF), xóa và khởi tạo lại — đây là mẫu chuẩn của Espressif.
2. **Màn hình trước Wi-Fi**: màn hình cần một khối bộ nhớ *liên tục* 115 KB có khả năng DMA. Wi-Fi cấp phát rất nhiều khối nhỏ làm phân mảnh heap; cấp phát framebuffer trước đảm bảo còn khối lớn liền mạch.
3. **Còi trước mạng**: đặt GPIO9 lên mức cao (im lặng) càng sớm càng tốt, trước khi có bất kỳ bản tin nào.
4. **Servo trước mạng**: khôi phục vị trí barrier từ NVS trước khi có lệnh từ web/RFID, tránh trạng thái không xác định.
5. **Mạng sau cùng**: chỉ nhận lệnh khi mọi thiết bị đã sẵn sàng.

`ESP_ERROR_CHECK` áp dụng nguyên tắc *fail-fast*: nếu khởi tạo thất bại, hệ thống in lỗi và khởi động lại thay vì chạy tiếp ở trạng thái nửa vời.

6.3 Các tác vụ FreeRTOS và đồng bộ

Bảng 6.1: Các tác vụ FreeRTOS trên máy chủ

Task	Ưu tiên	Stack	Chu kỳ	Nhiệm vụ
`udp_server`	5	4096 B	theo sự kiện	Chặn ở `recvfrom`, phân tích, cập nhật trạng thái, ra lệnh servo/còi
`servo_auto_close`	5	3072 B	50 ms	Đóng barrier khi đến hạn
`httpd` (ESP-IDF)	5	4096 B	theo request	Phục vụ trang web và API
`lcd_display`	4	4096 B	500 ms (120 ms khi cháy)	Vẽ lại màn hình khi trạng thái đổi
`lcd_animation`	3	3072 B	100 ms/khung	Hoạt hình GIF trang trí

Tại sao đặt ưu tiên như vậy. Ưu tiên phản ánh *mức độ quan trọng đối với người dùng*. Nhận lệnh và điều khiển thanh chắn (ưu tiên 5) liên quan trực tiếp đến xe đang chờ ở cổng nên phải được chạy ngay. Màn hình thông tin (4) quan trọng hơn hoạt hình trang trí (3): nếu CPU bận, hoạt hình chậm đi chứ thông tin không bị trễ. Kích thước stack 3–4 KB đủ cho các biến cục bộ (bộ đệm UDP 256 B, chuỗi định dạng) cộng với `ESP_LOG` (dùng `vprintf` khá tốn

stack); với `FREERTOS_HZ = 100`, tick 10 ms nên các chu kỳ đều là bội số của tick.

Hệ thống dùng **hai cơ chế đồng bộ khác nhau** một cách có chủ đích:

- `parking_state` và `rfid_session` dùng **critical section** (`portENTER_CRITICAL` với spinlock `portMUX_TYPE`). Vùng bảo vệ chỉ là vài lệnh `memcpy` trên cấu trúc vài chục byte — chạy trong vài micro giây. Critical section rẻ hơn mutex và an toàn kể cả nếu sau này gọi từ ngắt.
- `servo` và `lcd` dùng **mutex** (`xSemaphoreCreateMutex`). Vùng bảo vệ của servo bao gồm ghi NVS (ghi flash, có thể mất hàng mili giây) và vùng của LCD bao gồm gửi dữ liệu SPI. Không được làm việc lâu như vậy trong critical section vì nó tắt ngắt và chặn bộ lập lịch; mutex cho phép task khác vẫn chạy và có cơ chế kế thừa ưu tiên (priority inheritance) tránh đảo ưu tiên.

6.4 Module trạng thái bãi xe

```c
#define PARKING_SLOT_COUNT 5
typedef struct {
    int slots[PARKING_SLOT_COUNT]; /* 1 = occupied, 0 = free */
    float temperature_c;
    float water_mm;
    bool fire;
    bool received;              /* da nhan it nhat 1 ban tin */
    bool temperature_received;  /* nhiet do den tu RFID event */
} parking_state_t;
```

Mã nguồn 6.3: Cấu trúc trạng thái dùng chung (`parking_state.h`)

Các cờ `received` và `temperature_received` cho phép giao diện hiển thị "`--.-`" / "WAITING" thay vì số 0 giả khi chưa có dữ liệu — tránh hiểu lầm "mực nước 0 mm" khi thực ra cảm biến chưa kết nối. Hàm `parking_state_get` trả về *bản sao* và đếm số chỗ trống ngay trong critical section, nên người đọc luôn nhận được một ảnh nhất quán (không có trường hợp đọc nửa cũ nửa mới).

6.5 Bộ phân tích JSON tự viết

Tại sao không dùng thư viện cJSON. Gói UDP đến từ mạng là *ranh giới tin cậy*: bất kỳ thiết bị nào kết nối vào AP đều gửi được. cJSON cấp phát heap cho từng nút, có thể phân mảnh heap khi chạy dài ngày và chấp nhận mọi JSON hợp lệ (kể cả trường lạ, mảng 1000 phần tử). Bộ phân tích tự viết (khoảng 180 dòng) **không cấp phát động**, chỉ chấp nhận đúng lược đồ mong đợi và có **vòng lặp giới hạn** (tối đa 5 trường cho `parking_update`,

> 3 trường cho `rfid_event`) nên thời gian xử lý bị chặn trên, không thể bị làm treo bằng gói tin độc hại.

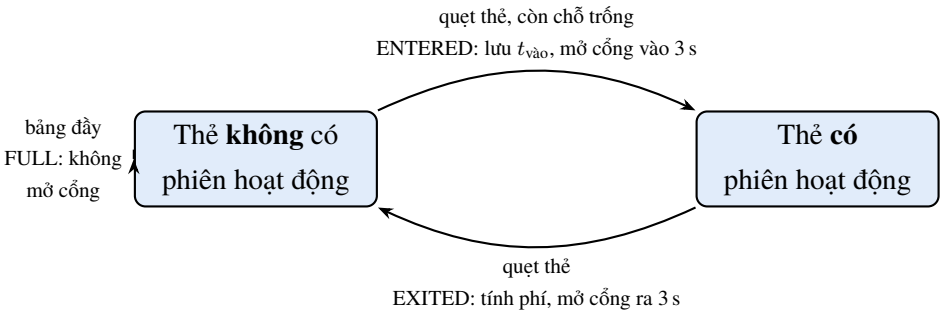
Các quy tắc kiểm tra:

- Khóa phải thuộc tập cho phép; khóa lạ hoặc lặp lại ⇒ từ chối.
- `slots` phải đúng 5 phần tử, mỗi phần tử chỉ là ký tự `0` hoặc `1`.
- `water_mm` trong $[0;\ 10\,000]$; `temperature_c` trong $[-40;\ 125]$ (dải đo của hầu hết cảm biến nhiệt độ bán dẫn, bao trùm DHT11); biểu thức `!(v >= a && v <= b)` còn loại được giá trị `NaN`.
- `fire` phải là JSON boolean (`true`/`false`), không chấp nhận `1`/`0`; được phép thiếu để tương thích client cũ.
- `card_id` chỉ gồm chữ, số, `-`, `_`, tối đa 31 ký tự (UID MIFARE dài nhất 10 byte = 20 ký tự hex), được chuẩn hóa thành chữ hoa để `a1b2` và `A1B2` là cùng một thẻ.
- Không được có dữ liệu thừa sau dấu `}` đóng.

Bộ đệm nhận có kích thước 256 byte và `recvfrom` chỉ đọc tối đa 255 byte, sau đó gán `buffer[received] = '\0'`, đảm bảo luôn là chuỗi kết thúc hợp lệ; gói dài hơn bị cắt và sẽ bị bộ phân tích từ chối. 256 byte gấp hơn 3 lần bản tin dài nhất (≈80 byte), đủ dư cho mở rộng.

6.6 Phiên gửi xe RFID và tính phí

Mỗi phiên gồm UID và thời điểm vào (`esp_timer_get_time()`, micro giây kể từ khi khởi động, 64 bit — không tràn trong hơn 290 000 năm). Bảng phiên có `RFID_MAX_ACTIVE = 5` phần tử, *bằng số chỗ đỗ*: không thể có nhiều xe trong bãi hơn số chỗ, đồng thời dùng mảng tĩnh thay vì danh sách liên kết cấp phát động.



Hình 6.1: Máy trạng thái của một thẻ RFID

Công thức tính phí với t là số giây gửi xe:

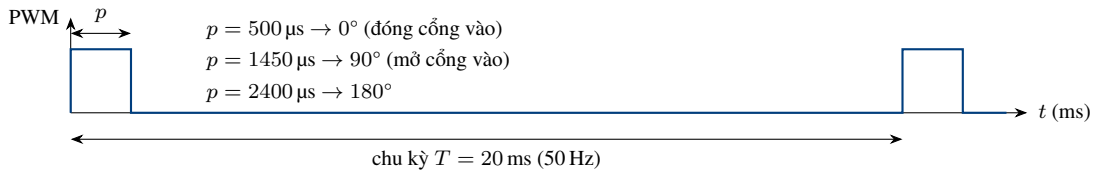
$$h = \max\left(1,\ \left\lceil \frac{t}{3600} \right\rceil\right), \qquad \text{Phí} = 5000 \cdot h \ \ (\text{VNĐ}) \tag{6.1}$$

Trong mã nguồn, phép làm tròn lên được viết bằng số nguyên `(t + 3600 - 1) / 3600` để tránh dùng số thực. Kết quả được bão hòa ở `UINT32_MAX` nếu tràn. Ví dụ: gửi 1 giờ 1 phút ($t = 3660$) $\Rightarrow h = 2 \Rightarrow$ 10.000 VNĐ; gửi 10 giây $\Rightarrow$ 5.000 VNĐ (tối thiểu).

Tại sao chọn các thông số này. **5.000 VNĐ/giờ** gần với mức phí gửi xe phổ biến tại Hà Nội; **làm tròn lên theo giờ, tối thiểu 1 giờ** là cách tính phí thông dụng của bãi xe và dễ giải thích cho người dùng. Hóa đơn hiển thị **60 giây** — đủ thời gian để tài xế đọc và trả tiền, sau đó màn hình trở về trạng thái bình thường. Barrier mở **3 giây** — đủ để một xe mô hình đi qua, nhưng không quá lâu để xe thứ hai "đi ké".

6.7 Điều khiển thanh chắn bằng LEDC PWM

Servo SG90 nhận xung chu kỳ 20 ms (50 Hz); độ rộng xung quyết định góc. Firmware dùng dải hiệu chỉnh 500–2400 µs cho 0–180° (giá trị đo thực tế của SG90; datasheet danh nghĩa là 1000–2000 µs nhưng thường không đạt đủ 180°).



Hình 6.2: Dạng xung điều khiển servo SG90

Từ góc θ, độ rộng xung và giá trị duty của bộ đếm 14 bit được tính:

$$p(\theta) = 500 + \frac{1900\,\theta}{180}\ \mu\text{s}, \qquad D = \left\lfloor \frac{p \cdot (2^{14} - 1)}{20\,000} \right\rfloor \tag{6.2}$$

Với $\theta = 90°$: $p = 1450$ µs, $D = 1187$; với $\theta = 0°$: $p = 500$ µs, $D = 409$.

Tại sao 50 Hz, 14 bit, LEDC. **50 Hz** là tần số chuẩn của servo RC analog; tần số cao hơn có thể làm nóng hoặc rung SG90. **14 bit** là độ phân giải lớn nhất của LEDC trên ESP32-C3: mỗi bước $= 20\,000/16\,384 \approx 1{,}22$ µs, tương ứng $\approx 0{,}12°$ — mịn hơn nhiều so với vùng chết của SG90 (khoảng 1°), nên servo không bị "rung tìm vị trí". **LEDC** là PWM phần cứng: sau khi đặt duty, xung được phát liên tục mà CPU không phải can thiệp, nên dù Wi-Fi hay màn hình đang bận thì servo vẫn giữ góc ổn định — điều PWM bằng phần mềm không đảm bảo được. Hai servo dùng chung một timer (cùng 50 Hz) nhưng hai kênh khác nhau.

```c
static esp_err_t servo_set_angle(ledc_channel_t channel, int angle)
{
    const uint32_t pulse_us = SERVO_MIN_PULSE_US +
        (SERVO_MAX_PULSE_US - SERVO_MIN_PULSE_US) * (uint32_t)angle /
    180;
    const uint32_t duty = pulse_us * ((1U << 14) - 1) / SERVO_PERIOD_US;
    esp_err_t err = ledc_set_duty(LEDC_LOW_SPEED_MODE, channel, duty);
    if (err != ESP_OK)
        return err;
    return ledc_update_duty(LEDC_LOW_SPEED_MODE, channel);
}

static int servo_angle_for_state(bool is_out_gate, bool open)
{
    /* The exit barrier is mounted in the opposite orientation. */
    if (is_out_gate)
        return open ? 0 : 90;
    return open ? 90 : 0;
}
```

Mã nguồn 6.4: Đặt góc servo và ánh xạ trạng thái (`servo.c`)

Một số quyết định thiết kế quan trọng:

- **Barrier ra lắp đối xứng** với barrier vào trên mô hình, nên góc mở/đóng bị đảo (0° = mở). Thay vì đảo dây hay đảo logic ở nhiều nơi, việc ánh xạ được gom vào một hàm duy nhất `servo_angle_for_state`.
- **Chỉ quay trong 0–90°**: một phần tư vòng là đủ nâng thanh chắn từ ngang lên dọc; quay ít hơn thì nhanh hơn và ít tải hơn.
- **Kênh LEDC khởi tạo với duty = 0** (không phát xung) cho đến khi đọc xong trạng thái từ NVS, sau đó mới đặt góc. Như vậy khi khởi động lại, servo đi thẳng đến vị trí đã lưu, không bị giật về một góc mặc định trước.
- **Chỉ ghi NVS khi trạng thái thay đổi** (`if (*state == open) return`), giảm số lần ghi flash (flash có tuổi thọ giới hạn số lần xóa). Nếu ghi NVS lỗi, servo vẫn được coi là đã chuyển động thành công vì trạng thái vật lý mới là điều quan trọng; lỗi chỉ được ghi log.

Tự động đóng. `servo_open_for()` mở barrier và ghi thời điểm hết hạn `now + 3000 ms`. Task `servo_auto_close` thức dậy mỗi 50 ms, so sánh với `esp_timer_get_time()` và đóng khi đến hạn.

Tại sao dùng task kiểm tra 50 ms thay vì software timer. Mỗi lần đóng cần ghi NVS, là thao tác có thể chặn — không nên thực hiện trong callback của `esp_timer` hay FreeRTOS timer (chạy trong task timer dùng chung, stack nhỏ). Task riêng có stack 3 KB và có thể lấy mutex an toàn. Sai số đóng tối đa 50 ms (5 tick) so với 3 s là khoảng 1,7%, người dùng không nhận thấy, mà CPU chỉ thức 20 lần/giây. Lệnh điều khiển thủ công từ web đặt lại hạn đóng về 0, tức là **lệnh của người vận hành luôn được ưu tiên** hơn lịch tự động.

6.8 Còi báo cháy

Còi được cấu hình là ngõ ra số, không kéo lên/xuống nội, và đặt mức cao ngay lập tức (im lặng). Mỗi khi một bản tin `parking_update` hợp lệ được chấp nhận, `buzzer_set_fire(new_state.fire)` được gọi. Còi kêu liên tục trong lúc bản tin mới nhất báo cháy, và tắt khi bản tin báo hết cháy.

6.9 Màn hình ST7789

6.9.1 Cấu hình SPI

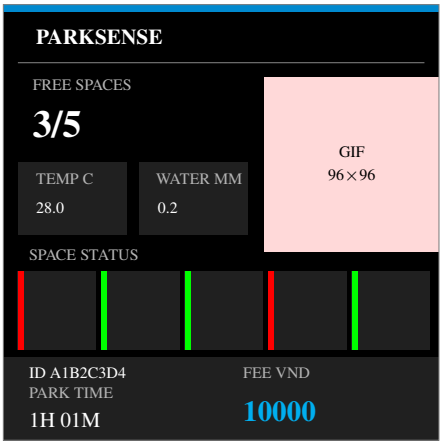
Tại sao SPI 40 MHz và DMA. Một khung hình đầy đủ là $240 \times 240 \times 16 = 921\,600$ bit. Ở 40 MHz, thời gian truyền lý thuyết là $921\,600/40 \cdot 10^6 \approx 23$ ms, tức tối đa khoảng 43 khung/giây — dư cho giao diện tĩnh và màn hình cảnh báo cháy 8 khung/giây. 40 MHz bằng đúng $80\,\text{MHz}/2$ (xung APB), là ước số nguyên nên không có sai số tần số, đồng thời nằm dưới giới hạn ghi của ST7789 (chu kỳ ghi tối thiểu khoảng 16 ns $\approx$ 62 MHz) với biên an toàn cho dây cắm breadboard. DMA được bật (`SPI_DMA_CH_AUTO`) để CPU không phải đẩy từng byte; hàng đợi giao dịch sâu 10 (`trans_queue_depth`) cho phép xếp nhiều lần vẽ liên tiếp.

6.9.2 Framebuffer và định dạng màu

Toàn bộ giao diện được vẽ vào một framebuffer RGB565 trong RAM (cấp phát với `MALLOC_CAP_DMA`), rồi gửi một lần bằng `esp_lcd_panel_draw_bitmap`.

- **RGB565** (16 bit/điểm ảnh) thay vì RGB666 (18 bit) giảm 1/3 băng thông và bộ nhớ, mắt người khó phân biệt trên màn 1,54 inch.
- **Đảo byte** (`rgb565_byte_swap`): ESP32-C3 là little-endian, trong khi ST7789 nhận byte cao trước qua SPI. Đảo một lần khi ghi vào framebuffer rẻ hơn cấu hình driver đảo từng giao dịch.
- **Đảo màu** (`esp_lcd_panel_invert_color(true)`): các module IPS ST7789 phổ biến cần lệnh INVON để hiển thị đúng màu (nếu không, đen thành trắng).

- **Font 5×7 tự định nghĩa** cho 36 ký tự A–Z, 0–9 (180 byte) cộng vài ký hiệu, có thể phóng to theo hệ số `scale`. Không cần thư viện đồ họa (LVGL tốn hàng trăm KB flash và RAM) cho một giao diện cố định.



Bố cục 240×240:
• Hàng 0–32: tiêu đề.
• Hàng 41–80: số chỗ trống, cỡ chữ ×3.
• Hàng 87–127: nhiệt độ, mực nước.
• Góc phải: ảnh động 96×96.
• Hàng 147–190: 5 ô chỗ đỗ, vạch màu đỏ/xanh.
• Hàng 194–240: hóa đơn sau khi xe ra (hoặc ảnh động nhỏ khi không có hóa đơn).

Hình 6.3: Bố cục giao diện màn hình TFT (mô phỏng)

6.9.3 Chỉ vẽ lại khi cần

Task `lcd_display` lưu bản sao của trạng thái lần vẽ trước và chỉ vẽ lại khi có khác biệt (chỗ đỗ, nhiệt độ, nước, cháy, hóa đơn). Ở trạng thái bình thường task thức mỗi 500 ms — đủ nhanh so với chu kỳ dữ liệu 1 s. Khi có cháy, chu kỳ rút xuống 120 ms (≈ 8 khung/giây) để ngọn lửa nhấp nháy, thu hút sự chú ý. Cách làm này tiết kiệm băng thông SPI và CPU cho các tác vụ mạng.

6.9.4 Hoạt hình GIF nén LZSS

Ảnh GIF trang trí được chuyển trước (trên máy tính) thành các khung RGB565, nén bằng LZSS và lưu dưới dạng mảng hằng trong flash. Mỗi khung được giải nén ngay trước khi hiển thị vào một bộ đệm DMA nhỏ.

Bảng 6.2: Hiệu quả nén khung hình

Ảnh	Kích thước	Số khung	Dữ liệu thô	Sau nén LZSS
Kaoruko	96×96	10	184 320 B	71 652 B (2,6×)
Claude	68×46	18	112 608 B	16 325 B (6,9×)

Tại sao nén LZSS thay vì lưu thô hoặc giải mã GIF trên chip. Lưu thô tốn gần 300 KB flash trong khi phân vùng ứng dụng chỉ có 1 MB; giải mã GIF (LZW + bảng màu) trên chip cần bảng từ điển lớn và mã phức tạp. LZSS có bộ giải nén chỉ khoảng 40 dòng, *không cần bộ nhớ phụ* (tham chiếu ngược ngay trong bộ đệm đầu ra), mỗi mã gồm 12 bit khoảng cách + 4 bit độ dài (3–18 điểm ảnh) rất hợp với ảnh có nhiều vùng màu đồng nhất. Bộ giải nén kiểm tra biên đầy đủ (`distance > output`, `length > pixel_count - output`) nên

> dữ liệu hỏng không gây ghi tràn bộ nhớ.

Hai bộ đệm (double buffering). `esp_lcd_panel_draw_bitmap` chỉ *xếp hàng* giao dịch DMA rồi trả về ngay. Nếu giải nén khung kế tiếp vào cùng bộ đệm khi DMA vẫn đang đọc, ảnh sẽ bị xé/rác. Vì vậy có hai bộ đệm $96 \times 96 \times 2 = 18\,432$ byte dùng luân phiên (`gif_buffer_index ^= 1`).

Xóa nền trắng. Ảnh gốc có nền trắng; thuật toán loang (flood fill) từ viền ảnh đánh dấu các điểm gần trắng liên thông với viền là nền và tô đen, nhưng giữ các chi tiết trắng nằm bên trong nhân vật. Thay vì hàng đợi BFS (tới 9216×2 byte), thuật toán dùng một **bitset 1 152 byte** và lặp lan truyền cho đến khi không còn thay đổi — đổi một ít thời gian CPU lấy bộ nhớ.

Mutex LCD. Hai task (`lcd_display` và `lcd_animation`) cùng ghi vào một panel; mutex đảm bảo các giao dịch vẽ không chen ngang nhau.

6.10 UDP server

```
for (;;) {
    int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_IP);
    if (sock < 0) { vTaskDelay(pdMS_TO_TICKS(1000)); continue; }
    bind(sock, /* INADDR_ANY:3333 */ ...);
    for (;;) {
        char buffer[256];
        int n = recvfrom(sock, buffer, sizeof(buffer) - 1, 0, ...);
        if (n < 0) break;                          /* tao lai socket */
        buffer[n] = '\0';
        parking_state_t s = {0};
        if (parking_update_parse_json(buffer, &s)) {
            parking_state_update(&s);
            buzzer_set_fire(s.fire);
            continue;
        }
        rfid_event_t e = {0};
        if (!rfid_event_parse_json(buffer, &e)) continue; /* bo qua */
        switch (rfid_session_scan(&e, esp_timer_get_time())) {
        case RFID_SCAN_ENTERED: servo_open_for(SERVO_GATE_IN, 3000);
    break;
        case RFID_SCAN_EXITED:  servo_open_for(SERVO_GATE_OUT, 3000);
    break;
        default: /* bai day */ break;
        }
    }
    close(sock);
```

```
25      vTaskDelay(pdMS_TO_TICKS(1000));
26  }
```

Mã nguồn 6.5: Vòng lặp chính của UDP server (rút gọn)

Task chặn ở `recvfrom` nên **không tiêu tốn CPU** khi không có gói tin. Hai vòng lặp lồng nhau tạo cơ chế **tự phục hồi**: nếu socket lỗi, task đóng socket, chờ 1 s (tránh vòng lặp lỗi liên tục chiếm CPU) rồi tạo lại. Bind vào `INADDR_ANY` để nhận cả gói unicast tới `192.168.4.1` lẫn gói broadcast.

6.11 HTTP server và API

Server dùng `esp_http_server` với `max_uri_handlers = 10` — đúng bằng số route đã đăng ký (mặc định của ESP-IDF là 8, không đủ). Các lệnh điều khiển dùng phương thức `POST` vì chúng làm thay đổi trạng thái (trình duyệt hay công cụ tìm kiếm có thể tự động gọi `GET` khi tải trước liên kết). Phản hồi JSON của `/api/parking` định dạng số thực bằng số nguyên "phần mười" (`(int)(v*10 + 0.5)`) để luôn có đúng một chữ số thập phân, xử lý đúng số âm và không phụ thuộc hỗ trợ `%f` của thư viện C trên vi điều khiển.

Chương 7. Thiết kế firmware nút cảm biến (ESP8266)

7.1 Sensor Client: cảm biến chỗ đỗ, lửa và nước

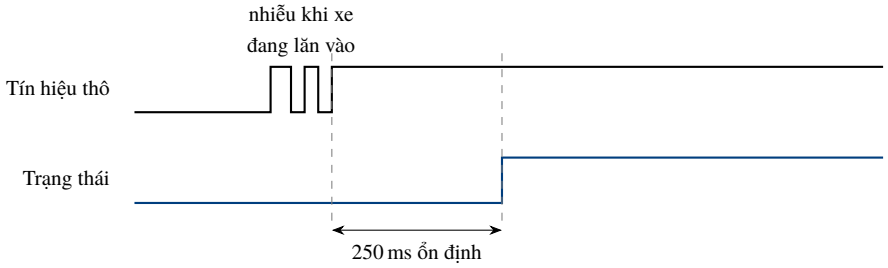
7.1.1 Vòng lặp không chặn

Toàn bộ chương trình dựa trên `millis()` thay vì `delay()`: mỗi vòng `loop()` gọi `ArduinoOTA.handle()`, đọc cảm biến, và chỉ gửi UDP khi đã đủ 1 s kể từ lần gửi trước. Như vậy cảm biến được lấy mẫu liên tục (hàng nghìn lần mỗi giây), OTA luôn phản hồi và Wi-Fi stack của ESP8266 (cần được nhường CPU thường xuyên) không bị bỏ đói. Phép so sánh dạng `now - last >= interval` với số không dấu vẫn đúng khi `millis()` tràn sau khoảng 49,7 ngày.

7.1.2 Chống rung (debounce) 250 ms

```
int current = read_active_low(sensorPins[i]);
if (current != pendingState[i]) {      // tin hieu vua thay doi
    pendingState[i] = current;
    changeTime[i] = now;               // bat dau dem lai
}
if (pendingState[i] != state[i] &&
    now - changeTime[i] >= STABLE_TIME) {
    state[i] = pendingState[i];        // on dinh du 250 ms -> chap nhan
}
```

Mã nguồn 7.1: Thuật toán chống rung cho cảm biến IR (`SensorClient.ino`)



Hình 7.1: Nguyên lý chống rung: chỉ chấp nhận mức mới khi giữ ổn định đủ 250 ms

Tại sao 250 ms. Khi xe đang tiến vào chỗ, mép xe và bánh xe làm tín hiệu IR bật/tắt liên tục; cảm biến lửa cũng nhấp nháy theo ngọn lửa. Thời gian ổn định phải *lớn hơn* thời gian nhiễu này (thường vài chục đến hơn trăm mili giây) nhưng *nhỏ hơn đáng kể* chu kỳ gửi 1 s để không làm tăng độ trễ cảm nhận. 250 ms là điểm cân bằng nhóm đã tinh chỉnh thực nghiệm (commit `update stable_time`). Mỗi cảm biến có bộ đếm riêng nên nhiễu ở chỗ này không ảnh hưởng chỗ khác.

7.1.3 Logic tích cực mức thấp và cảm biến lửa

Module IR kéo ngõ ra xuống mức thấp khi phát hiện vật, nên hàm `read_active_low()` trả về `!digitalRead(pin)`, chuẩn hóa thành "1 = có xe". Với cảm biến lửa, firmware gửi `fire = true` khi chân DO ở *mức cao* (`flameState ? "false" : "true"`). Theo lịch sử commit (`reverse signal`), đây là kết quả hiệu chỉnh thực nghiệm với module lửa mà nhóm sử dụng (ngưỡng biến trở và mức logic của module đảo so với IR). Khi thay module khác cần kiểm tra lại mức logic này.

7.1.4 Quy đổi mực nước

ADC của ESP8266 là 10 bit (0–1023). Giá trị được quy đổi tuyến tính sang milimét:

$$h_{\text{mm}} = \begin{cases} 0, & raw \le 20 \\ \dfrac{\min(raw, 700) - 20}{700 - 20} \cdot 40, & raw > 20 \end{cases} \tag{7.1}$$

Tại sao các hằng số 20, 700, 40, 250 ms. `WATER_DRY_ADC = 20` là mức nhiễu đo được khi cảm biến khô — dưới mức này coi là 0 để màn hình không nhảy 0.1, 0.2 vô nghĩa. `WATER_MAX_ADC = 700` là giá trị khi nhúng ngập hoàn toàn (không đạt 1023 do điện trở nước và mạch chia áp của module). `40 mm` là chiều dài vùng đo của loại cảm biến điện trở phổ biến. Hàm `constrain` đảm bảo kết quả không vượt 40 mm. Đọc ADC mỗi 250 ms thay vì mỗi vòng lặp vì trên ESP8266 gọi `analogRead` quá dày có thể làm ảnh hưởng tới Wi-Fi (ADC dùng chung tài nguyên với RF), trong khi mực nước thay đổi chậm.

7.1.5 Đóng gói và gửi

```
char payload[112];
int length = snprintf(payload, sizeof(payload),
  "{\"type\":\"parking_update\",\"slots\":[%d,%d,%d,%d,%d],"
  "\"water_mm\":%.1f,\"fire\":%s}",
  state[0], state[1], state[2], state[3], state[4],
  currentWaterMm, flameState ? "false" : "true");
if (length < 0 || length >= sizeof(payload)) return;  // khong gui ban tin cut cut
```

Mã nguồn 7.2: Tạo bản tin `parking_update`

Bộ đệm tĩnh trên stack, `snprintf` giới hạn độ dài và kiểm tra giá trị trả về nên không bao giờ gửi một bản tin bị cắt dở. Trước khi gửi, kiểm tra `WiFi.status()`; `setAutoReconnect(true)` giúp tự nối lại khi máy chủ khởi động lại.

Tại sao chu kỳ gửi 1 s. 1 s khớp với chu kỳ làm mới của dashboard (cũng 1 s), đảm bảo yêu cầu độ trễ ≤ 2 s kể cả khi mất một gói. Gửi nhanh hơn (ví dụ 100 ms) không mang lại lợi ích cảm nhận được với chỗ đỗ xe nhưng tăng 10 lần lưu lượng broadcast (broadcast Wi-Fi truyền ở tốc độ cơ sở thấp nên tốn thời gian kênh) và tăng tải cho máy chủ.

7.2 RFID Client: hai đầu đọc RFID và DHT11

7.2.1 Ghép hai MFRC522 trên một bus SPI

Hai đầu đọc dùng chung SCK/MOSI/MISO và chân RST, mỗi đầu có chân SS riêng. Khi khởi tạo, cả hai SS được kéo lên cao (không chọn), sau đó lần lượt `PCD_Init`, đặt **độ khuếch đại ăng-ten tối đa 48 dB** để tăng tầm đọc và **tắt ăng-ten**.

```
for (uint8_t reader = 0; reader < NR_OF_READERS; reader++) {
    mfrc522[reader].PCD_AntennaOn();
    delay(4);  // cho tu truong nap nang luong cho the
    if (mfrc522[reader].PICC_IsNewCardPresent() &&
        mfrc522[reader].PICC_ReadCardSerial()) {
        String uid = getCardUID(mfrc522[reader].uid.uidByte,
                                mfrc522[reader].uid.size);
        if (uid != lastUID[reader] ||
            millis() - lastReadTime[reader] >= COOLDOWN_TIME)
            sendUDPRfid(uid);
        mfrc522[reader].PICC_HaltA();
        mfrc522[reader].PCD_StopCrypto1();
    }
    mfrc522[reader].PCD_AntennaOff();  // nhuong cho dau doc kia
    yield();
}
```

Mã nguồn 7.3: Quét luân phiên hai đầu đọc (`RFIDClient.ino`)

Tại sao bật/tắt ăng-ten luân phiên, 4 ms, 48 dB, 1,5 s. Hai đầu đọc cùng phát sóng 13,56 MHz đặt gần nhau sẽ **nhiễu chéo**: thẻ đặt ở cổng vào có thể bị cả hai đầu đọc cùng cấp năng lượng và trả lời lẫn lộn. Chỉ bật một ăng-ten tại một thời điểm loại bỏ hoàn toàn vấn đề này. Thẻ thụ động cần vài mili giây để nhận đủ năng lượng từ trường và khởi động; **4 ms** là đủ cho thẻ MIFARE mà vẫn giữ chu kỳ quét hai đầu đọc ngắn (vài chục ms), người dùng không cảm nhận được độ trễ. Gain **48 dB** bù lại tầm đọc bị giảm khi đặt đầu đọc sau lớp vỏ mô hình. Khi thẻ được giữ yên trên đầu đọc, nó sẽ bị đọc lại liên tục; thời gian chờ **1,5 s** cho cùng một thẻ trên cùng một đầu đọc ngăn gửi nhiều sự kiện, vì với logic chuyển trạng thái vào/ra, một lần quẹt bị gửi hai lần sẽ biến "vào" thành "ra" ngay lập tức. 1,5 s đủ dài để người dùng nhấc thẻ ra, nhưng ngắn hơn 3 s barrier mở. `PICC_HaltA` và `PCD_`

> `StopCrypto1` đưa thẻ về trạng thái nghỉ đúng chuẩn ISO 14443A.

UID được chuyển thành chuỗi hex in hoa, mỗi byte luôn 2 ký tự (thêm `0` đứng trước nếu nhỏ hơn `0x10`) để cùng một thẻ luôn cho cùng một chuỗi.

7.2.2 Cảm biến nhiệt độ DHT11

> **Tại sao đọc DHT11 mỗi 5 s**. DHT11 yêu cầu khoảng cách giữa hai lần đọc tối thiểu 1 s, và mỗi lần đọc giao thức 1-wire mất khoảng 20–25 ms với ngắt bị tắt — có thể làm lỡ thẻ RFID hoặc gói Wi-Fi. Nhiệt độ môi trường thay đổi chậm, nên 5 s là chu kỳ hợp lý. Kết quả `NaN` (đọc lỗi) bị bỏ qua và giữ giá trị cũ, không gửi số sai cho máy chủ. Nhiệt độ được gửi kèm mỗi sự kiện RFID, nên máy chủ cập nhật nhiệt độ mỗi khi có xe ra vào.

7.2.3 Nạp firmware qua mạng (OTA)

Cả hai client gọi `ArduinoOTA.begin()` và `handle()` trong vòng lặp. Sau khi gắn bo vào mô hình, nhóm có thể nạp firmware mới qua Wi-Fi `ESP32-Parking` từ Arduino IDE mà không phải tháo bo. Trong lúc OTA, ăng-ten RFID được tắt để giảm nhiễu và tải.

Chương 8. Giao thức truyền thông và giao diện web

8.1 Thông số mạng

Bảng 8.1: Thông số mạng

SSID / mật khẩu	`ESP32-Parking` / `12345678` (WPA2-PSK)
Kênh / số trạm tối đa	1 / 4
Địa chỉ máy chủ	`192.168.4.1` (DHCP server cấp IP cho client)
Cổng UDP	3333, client gửi tới `192.168.4.255:3333`
Web	`http://192.168.4.1` (cổng 80)

8.2 Bản tin UDP

```
{"type":"parking_update","slots":[1,0,0,1,0],"water_mm":0.2,"fire":false}
```

Mã nguồn 8.1: Bản tin `parking_update` (Sensor Client → Server, mỗi 1 s)

```
{"type":"rfid_event","card_id":"A1B2C3D4","temperature_c":28.0}
```

Mã nguồn 8.2: Bản tin `rfid_event` (RFID Client → Server, mỗi lần quẹt thẻ)

Bảng 8.2: Đặc tả trường dữ liệu

Trường	**Kiểu**	**Ràng buộc**
`type`	chuỗi	`"parking_update"` hoặc `"rfid_event"`
`slots`	mảng 5 số	mỗi phần tử 0 (trống) hoặc 1 (có xe)
`water_mm`	số	$0 \le x \le 10\,000$
`fire`	boolean	tùy chọn; `true` = có cháy
`card_id`	chuỗi	1–31 ký tự `[A-Za-z0-9_-]`, chuẩn hóa in hoa
`temperature_c`	số	$-40 \le x \le 125$

8.3 HTTP API

Bảng 8.3: Các endpoint HTTP

Phương thức	Đường dẫn	Chức năng
GET	`/`, `/style.css`	Trang dashboard
GET	`/api/parking`	Trạng thái: `received`, `slots`, `free`, `total`, `temperature_c`, `water_mm`, `fire`
GET	`/api/barrier`	`{"in_open":bool,"out_open":bool}`
POST	`/api/barrier/in/open|close`	Mở/đóng barrier vào
POST	`/api/barrier/out/open|close`	Mở/đóng barrier ra
POST	`/api/barrier/all/open|close`	Khẩn cấp: mở/đóng cả hai

8.4 Dashboard

Trang web là HTML/CSS/JavaScript thuần (không framework) để kích thước nhỏ và không phụ thuộc CDN — điện thoại kết nối vào AP không có Internet. JavaScript gọi `fetch('/api/parking')` mỗi 1 s với `cache: 'no-store'` để không nhận dữ liệu cũ từ bộ đệm trình duyệt. Nếu request lỗi, trang hiển thị "ESP32 / OFFLINE". Khi người dùng bấm nút điều khiển barrier, các nút bị vô hiệu hóa cho tới khi có phản hồi, tránh gửi lệnh trùng. Nút khẩn cấp mở cả hai barrier phục vụ tình huống sơ tán khi có cháy.

> **Tại sao polling 1 s thay vì WebSocket.** Polling không giữ trạng thái kết nối trên máy chủ: mỗi request độc lập, xử lý xong giải phóng ngay — phù hợp với giới hạn socket và RAM của vi điều khiển, và đơn giản khi có nhiều trình duyệt cùng xem. Với dữ liệu thay đổi theo giây, độ trễ tối đa thêm 1 s là chấp nhận được.

Chương 9. Phân tích tài nguyên

9.1 Bộ nhớ RAM máy chủ

Bảng 9.1: Các vùng nhớ lớn do ứng dụng cấp phát (ước tính từ mã nguồn)

Thành phần	**Kích thước**
Framebuffer màn hình $240 \times 240 \times 2$ (DMA)	115 200 B
2 bộ đệm GIF $96 \times 96 \times 2$ (DMA)	36 864 B
Bitset xóa nền	1 152 B
Stack các task ứng dụng ($4096 \times 2 + 3072 \times 2$)	14 336 B
Bảng phiên RFID (5 phiên) + hóa đơn	$<$ 400 B
Tổng phần ứng dụng	$\approx$ **168 KB**

ESP32-C3 có 400 KB SRAM; sau khi trừ phần dành cho cache, ROM và hệ điều hành, heap khả dụng ban đầu khoảng 300 KB. Wi-Fi + lwIP + HTTP server cần thêm khoảng 60–100 KB. Tổng vẫn nằm trong giới hạn, nhưng framebuffer chiếm phần lớn — đây là lý do màn hình được khởi tạo trước Wi-Fi (xem trình tự khởi động ở Chương 6). Nếu cần thêm RAM, có thể vẽ theo dải (ví dụ 240×40) thay vì cả khung.

9.2 Bộ nhớ flash máy chủ

Bo dùng flash 2 MB với bảng phân vùng "single app" (phân vùng `factory` 1 MB). Bản build ngày 23/10/2026 có kích thước `.bin` là 1 025 888 byte, tức khoảng **97,8%** phân vùng ứng dụng; phần lớn là dữ liệu ảnh động ($\approx$ 88 KB sau nén) và các thư viện Wi-Fi/lwIP/HTTP. Đây là điểm cần lưu ý: thêm tính năng lớn sẽ vượt phân vùng. Giải pháp là dùng bảng phân vùng tùy chỉnh (ứng dụng $\approx$1,9 MB trên flash 2 MB) hoặc bỏ bớt khung ảnh động.

9.3 Thời gian và băng thông

- Truyền một khung màn hình đầy đủ: $\approx$23 ms ở 40 MHz.
- Khung GIF 96×96: 147 456 bit $\Rightarrow \approx$3,7 ms.
- Lưu lượng UDP từ Sensor Client: $\approx$75 byte/s — không đáng kể.
- Độ trễ cảm nhận chỗ đỗ: 250 ms (debounce) + tối đa 1 s (chu kỳ gửi) + tối đa 500 ms (chu kỳ vẽ) $\approx$ **tối đa 1,75 s**, đạt NFR1.

- Độ trễ RFID $\rightarrow$ barrier: thời gian quét (< 50 ms) + truyền UDP (vài ms) + xử lý (< 1 ms) + servo quay 90° (SG90 $\approx 0{,}1$ s/60° $\Rightarrow \approx 0{,}15$ s) $\approx$ **dưới 0,3 s**.

Chương 10. Kiểm thử

10.1 Kiểm thử đơn vị trên máy tính

Vì các module trong `Server/main/parking/` không phụ thuộc phần cứng (chỉ dùng `portENTER_CRITICAL` và `esp_timer_get_time`), chúng có thể được biên dịch trên máy tính với hai header giả lập (stub) và kiểm tra bằng `assert`. Bộ kiểm thử gồm 26 phép kiểm tra, biên dịch bằng `cc -std=c11 -Wall -Wextra` không có cảnh báo; **kết quả: 26/26 đạt**.

Bảng 10.1: Các trường hợp kiểm thử đơn vị

#	Đầu vào	Kỳ vọng / Kết quả
1	`parking_update` chuẩn của Sensor Client	Chấp nhận, đúng slot — Đạt
2	Thứ tự trường đảo, có khoảng trắng, `fire:true`	Chấp nhận — Đạt
3	Thiếu trường `fire` (client cũ)	Chấp nhận — Đạt
4	`slots` chỉ có 4 phần tử	Từ chối — Đạt
5	Slot có giá trị 2	Từ chối — Đạt
6	`water_mm` âm	Từ chối — Đạt
7	`fire: 1` (không phải boolean)	Từ chối — Đạt
8	Trường lạ `temperature_c` trong `parking_update`	Từ chối — Đạt
9	Khóa `type` bị lặp	Từ chối — Đạt
10	Dữ liệu thừa sau `}`	Từ chối — Đạt
11	`rfid_event` với UID chữ thường	Chấp nhận, chuẩn hóa `A1B2C3D4` — Đạt
12	UID có dấu cách	Từ chối — Đạt
13	Nhiệt độ 200 °C	Từ chối — Đạt
14	Thiếu `temperature_c`	Từ chối — Đạt
15–17	Thẻ vào, ra sau 3660 s	ENTERED, EXITED, phí 10.000 VNĐ, thời gian 3660 s — Đạt
18–20	Thẻ vào, ra sau 10 s	ENTERED, EXITED, phí tối thiểu 5.000 VNĐ — Đạt
21–25	Năm thẻ khác nhau vào liên tiếp	Cả 5 ENTERED — Đạt
26	Thẻ thứ sáu khi bãi đầy	FULL — Đạt

10.2 Kiểm thử tích hợp bằng công cụ giả lập

Thư mục `examples/` chứa các chương trình gửi bản tin giả lập, cho phép kiểm thử máy chủ khi chưa có nút cảm biến:

- `udp_sender.py`: gửi một `parking_update`.
- `udp_rfid_sender.py <UID>`: gửi một `rfid_event`; chạy hai lần cùng UID để giả lập vào rồi ra.
- `udp_sender_espidf.c`: ví dụ gửi từ một bo ESP-IDF khác.

Máy tính kết nối vào AP `ESP32-Parking`, chạy các script và quan sát log qua `idf.py monitor`, màn hình TFT và dashboard.

10.3 Kịch bản kiểm thử hệ thống

Bảng 10.2: Kịch bản kiểm thử trên phần cứng và hành vi mong đợi

#	Thao tác	Hành vi mong đợi
S1	Cấp nguồn máy chủ	AP `ESP32-Parking` xuất hiện; TFT hiện giao diện, các giá trị `--.-`; barrier về vị trí đã lưu
S2	Cấp nguồn Sensor Client	Sau khi kết nối, TFT/dashboard hiển thị 5/5 FREE, mực nước
S3	Đặt vật cản trước IR số 2	Trong ≤ 2 s, ô S2 chuyển BUSY (đỏ), FREE SPACES giảm 1
S4	Rê vật nhanh qua IR (< 250 ms)	Trạng thái không đổi (chống rung)
S5	Quẹt thẻ mới ở đầu đọc 1	Barrier vào mở, tự đóng sau $\approx$3 s; nhiệt độ cập nhật
S6	Giữ thẻ trên đầu đọc 1 s	Chỉ một sự kiện được gửi (cooldown 1,5 s)
S7	Quẹt lại cùng thẻ	Barrier ra mở; footer TFT hiện ID, thời gian, phí; hóa đơn biến mất sau 60 s
S8	Quẹt thẻ thứ 6 khi đã có 5 phiên	Không mở barrier; log cảnh báo bãi đầy
S9	Đưa bật lửa gần cảm biến lửa	Màn hình DANGER/FIRE DETECTED có lửa động, còi kêu, dashboard hiện băng rôn đỏ
S10	Tắt lửa	Sau debounce và gói kế tiếp, còi tắt, màn hình trở lại bình thường
S11	Nhúng cảm biến nước	Giá trị mực nước tăng, tối đa 40 mm

#	Thao tác	Hành vi mong đợi
S12	Bấm OPEN/CLOSE trên web	Barrier tương ứng chuyển động; trạng thái hiển thị đúng
S13	Mở barrier từ web rồi rút nguồn, cấp lại	Barrier khôi phục trạng thái mở (NVS)
S14	Tắt máy chủ khi đang mở web	Dashboard báo ESP32 / OFFLINE; Sensor Client tự kết nối lại khi máy chủ bật
S15	Gửi gói UDP rác từ máy tính	Máy chủ ghi log “Ignoring invalid” và tiếp tục hoạt động

Chương 11. Đánh giá, hạn chế và hướng phát triển

11.1 Kết quả đạt được

Hệ thống đáp ứng các yêu cầu chức năng FR1–FR10: phát hiện chỗ đỗ, ra vào bằng RFID với barrier tự đóng, tính phí, cảnh báo cháy bằng màn hình và còi, giám sát nước và nhiệt độ, dashboard điều khiển, lưu trạng thái barrier qua khởi động lại. Về kỹ thuật, nhóm đã vận dụng các kiến thức trọng tâm của lập trình nhúng: lập trình đa tác vụ với FreeRTOS và các cơ chế đồng bộ phù hợp, PWM phần cứng, SPI với DMA, quản lý bộ nhớ hạn chế (framebuffer, nén dữ liệu, tránh cấp phát động), lưu trữ không mất dữ liệu, lập trình socket và xử lý đầu vào không tin cậy.

11.2 Hạn chế

1. **Bảo mật**: bản tin UDP không được xác thực — bất kỳ thiết bị nào biết mật khẩu Wi-Fi đều có thể giả mạo thẻ RFID, báo cháy giả hoặc gọi API mở barrier (HTTP không có đăng nhập). Mật khẩu Wi-Fi cố định trong mã nguồn.
2. **Sự kiện RFID có thể mất**: khác với `parking_update`, sự kiện RFID không được gửi lại định kỳ; nếu gói broadcast bị mất, xe phải quẹt lại. Không có cơ chế xác nhận (ACK).
3. **Phiên gửi xe lưu trong RAM**: máy chủ khởi động lại thì mất toàn bộ phiên và hóa đơn; thời gian là thời gian kể từ khi khởi động, không phải giờ thực.
4. **Hướng vào/ra xác định theo phiên**, không theo đầu đọc: quẹt thẻ ở đầu đọc cổng ra khi chưa vào vẫn được xem là "vào".
5. **Nhiệt độ chỉ cập nhật khi có xe quẹt thẻ** vì đi kèm bản tin RFID.
6. **Số phiên RFID độc lập với cảm biến IR**: hai nguồn thông tin về số xe chưa được đối chiếu.
7. **Flash gần đầy** ($\approx$ 98% phân vùng ứng dụng).
8. **GPIO9 là chân strapping**: module còi không phù hợp có thể khiến bo vào chế độ nạp khi khởi động.

11.3 Hướng phát triển

- Thêm mã xác thực HMAC-SHA256 với khóa bí mật chung cho mỗi bản tin UDP (có sẵn trong mbedTLS của ESP-IDF) và số thứ tự chống phát lại; thêm đăng nhập cho API điều khiển barrier.

- Thêm bản tin ACK cho `rfid_event`, client gửi lại nếu không nhận được.
- Lưu phiên gửi xe vào NVS; đồng bộ giờ thực bằng SNTP (khi có Internet) hoặc module RTC DS3231.
- Gửi kèm chỉ số đầu đọc trong `rfid_event` để phân biệt cổng vào/ra.
- Kết nối MQTT lên đám mây để quản lý nhiều bãi, thống kê doanh thu, đặt chỗ trước qua ứng dụng; thanh toán bằng mã QR.
- Dùng bảng phân vùng tùy chỉnh và hỗ trợ OTA cho chính máy chủ.
- Thay cảm biến IR bằng cảm biến siêu âm hoặc từ trường để chính xác hơn với xe thật; thêm cảm biến khói MQ-2 bổ sung cho cảm biến lửa.

Kết luận

Đề tài đã xây dựng thành công một hệ thống bãi đỗ xe thông minh hoàn chỉnh ở quy mô mô hình với ba vi điều khiển giá rẻ, tự vận hành trên mạng Wi-Fi riêng. Mỗi lựa chọn — từ vi điều khiển, giao thức UDP, bộ phân tích JSON không cấp phát động, PWM 14 bit 50 Hz, SPI 40 MHz với DMA, đến các thông số thời gian 250 ms, 1 s, 1,5 s, 3 s — đều được cân nhắc dựa trên giới hạn tài nguyên của hệ thống nhúng và trải nghiệm của người dùng. Qua đề tài, nhóm đã hiểu sâu hơn về thiết kế phần mềm nhúng thời gian thực, giao tiếp ngoại vi và mạng, cũng như cách cân bằng giữa tính năng, độ tin cậy và tài nguyên.

Tài liệu tham khảo

[1] Espressif Systems, *ESP32-C3 Series Datasheet* và *ESP32-C3 Technical Reference Manual*.

[2] Espressif Systems, *ESP-IDF Programming Guide* (v6.1): Wi-Fi, LEDC, SPI Master, LCD, NVS, HTTP Server, FreeRTOS.

[3] Espressif Systems, *ESP8266EX Datasheet*.

[4] ESP8266 Community, *ESP8266 Arduino Core Documentation*.

[5] Sitronix, *ST7789V Datasheet — 240RGB×320 dot 262K Color TFT Controller*.

[6] NXP Semiconductors, *MFRC522 Standard Performance MIFARE and NTAG Frontend*, Product Data Sheet.

[7] TowerPro, *SG90 Micro Servo Datasheet*.

[8] Aosong, *DHT11 Humidity & Temperature Sensor Datasheet*.

[9] R. Barry, *Mastering the FreeRTOS Real Time Kernel*, Real Time Engineers Ltd.

[10] J. A. Storer, T. G. Szymanski, "Data compression via textual substitution", *Journal of the ACM*, 29(4), 1982.

[11] Thư viện *MFRC522* (miguelbalboa) và *DHT sensor library* (Adafruit) cho Arduino.

Chương A. Hướng dẫn biên dịch và nạp chương trình

Máy chủ ESP32-C3

Cài ESP-IDF 6.1 cùng toolchain ESP32-C3, mở terminal đã cấu hình ESP-IDF tại thư mục gốc của kho mã:

```
idf.py set-target esp32c3
idf.py build
idf.py -p <CONG_SERIAL> flash monitor
```

Đổi chân màn hình (nếu cần): `idf.py menuconfig` → *Parking TFT display*.

Nút cảm biến ESP8266

Trong Arduino IDE: cài ESP8266 core, cài thư viện `MFRC522` và `DHT sensor library` (cho RFID Client), mở sketch `SensorClient.ino` (thư mục `SensorClient/`) hoặc `RFIDClient.ino` (thư mục `RFIDClient/`), chọn bo ESP8266 và cổng, bấm Upload. Các lần sau có thể nạp qua mạng (cổng mạng OTA xuất hiện trong menu Port khi máy tính kết nối `ESP32-Parking`).

Giả lập không cần phần cứng

```
python examples/udp_sender.py
python examples/udp_rfid_sender.py A1B2C3D4   # chay 2 lan: vao roi ra
```

Chương B. Phân công công việc

Bảng B.1: Phân công công việc (nhóm điền)

Thành viên	Công việc
[Thành viên 1]	Firmware máy chủ: Wi-Fi AP, UDP/HTTP server, màn hình ST7789, dashboard
[Thành viên 2]	Firmware Sensor Client: cảm biến IR, lửa, nước; hiệu chỉnh debounce
[Thành viên 3]	Firmware RFID Client: RFID MFRC522, DHT11, OTA
[Thành viên 4]	Barrier servo, còi báo cháy, phiên RFID và tính phí, tài liệu